

```

}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.1.4 节例题 2
注意：几何计算的代码请充分利用常见的模板函数
同类题目：抱歉，HDU 1418
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表6-1所示。

表6-1 二维几何基础例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 6-1	UVa 11178	Morley's Theorem	线段相交；角度计算	刘汝佳
例 6-2	POJ 2284	That Nice Euler Circuit	线段相交；欧拉定理	刘汝佳

6.2 与圆有关的计算问题

本节通过一些经典题目的实现代码，提供了与圆有关的问题的算法实现。

例6-3 【圆和直线的各种问题】 二维几何110合一 (2D Geometry 110 in 1, UVa 12304)

这是一个拥有110₂个子问题的2D几何问题集。

- ☐ CircumscribedCircle $x_1 y_1 x_2 y_2 x_3 y_3$: 求三角形 (x_1, y_1) - (x_2, y_2) - (x_3, y_3) 的外接圆。这3点保证不共线。答案应格式化成 (x, y, r) ，表示圆心为 (x, y) ，半径为 r 。
- ☐ InscribedCircle $x_1 y_1 x_2 y_2 x_3 y_3$: 求三角形 (x_1, y_1) - (x_2, y_2) - (x_3, y_3) 的内切圆。这3点保证不共线。答案应格式化成 (x, y, r) ，表示圆心为 (x, y) ，半径为 r 。
- ☐ TangentLineThroughPoint $x_c y_c r x_p y_p$: 给定一个圆心在 (x_c, y_c) ，半径为 r 的圆，求过点 (x_p, y_p) 并且和这个圆相切

的所有切线。每条切线格式化为 angle ，表示直线的极角（角度， $0 \leq \text{angle} < 180$ ）。整个答案应格式化为列表（参见原题）。如果无解，应打印空列表。

□ CircleThroughAPointAndTangentToALineWithRadius $x_p y_p x_1 y_1 x_2 y_2 r$: 求出所有经过点 (x_p, y_p) 并且和直线 $(x_1, y_1)-(x_2, y_2)$ 相切的半径为 r 的圆。每个圆格式化为 (x, y) ，因为半径已经给定。整个答案应格式化为列表。

□ CircleTangentToTwoLinesWithRadius $x_1 y_1 x_2 y_2 x_3 y_3 x_4 y_4 r$: 给出两条不平行直线 $(x_1, y_1)-(x_2, y_2)$ 和 $(x_3, y_3)-(x_4, y_4)$ ，求所有半径为 r 并且同时和这两条直线相切的圆。每个圆格式化为 (x, y) ，因为半径已经给定。整个答案应格式化为列表。

□ CircleTangentToTwoDisjointCirclesWithRadius $x_1 y_1 r_1 x_2 y_2 r_2 r$: 给定两个相离的圆 (x_1, y_1, r_1) 和 (x_2, y_2, r_2) ，求出所有和这两个圆外切并且半径为 r 的圆。注意，因为是外切，求出的圆不能把这两个给定圆包含在内部。每个圆格式化为 (x, y) ，因为半径已经给定。整个答案应格式化为列表。

对于上述所有直线，输入的两个点应保证不重合。当格式化实数列表时，所有数应从小到大排列；当格式化二元组 (x, y) 时，先按 x 从小到大排序，当 x 相同时按 y 从小到大排序。

输入包含不超过1000个子问题，每个占一行，格式同题目描述。所有坐标均为绝对值不超过1000的整数，输入结束标志为文件结束符（EOF）。

对于输入的每行，按题目要求格式化输出。每个实数保留小数点后6位。列表用方括号，元组用圆括号。每行的输出中不应有空白符。

【代码实现】

```
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

const double eps = 1e-6;
int dcmp(double x) {
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}

const double PI = acos(-1);

struct Point {
    double x, y;
    Point(double x = 0, double y = 0): x(x), y(y) { }
    Point& operator=(const Point& p) {
        x = p.x, y = p.y;
        return *this;
    }
};

istream& operator>>(istream& is, Point& p) { return is >> p.x >> p.y; }
typedef Point Vector;

Vector operator + (const Vector& A, const Vector& B)
{ return Vector(A.x + B.x, A.y + B.y); }
Vector operator - (const Point& A, const Point& B)
{ return Vector(A.x - B.x, A.y - B.y); }
Vector operator * (const Vector& A, double p)
{ return Vector(A.x * p, A.y * p); }
Vector operator / (const Vector& A, double p)
{ return Vector(A.x / p, A.y / p); }
bool operator < (const Point& a, const Point& b) {
    return a.x < b.x || (a.x == b.x && a.y < b.y);
}
bool operator == (const Point& a, const Point &b) {
    return dcmp(a.x - b.x) == 0 && dcmp(a.y - b.y) == 0;
}
double Dot(const Vector& A, const Vector& B)
{ return A.x * B.x + A.y * B.y; }
double Length(const Vector& A) { return sqrt(Dot(A, A)); }
```

```

double Angle(const Vector& A, const Vector& B)
{ return acos(Dot(A, B) / Length(A) / Length(B)); }
double Cross(const Vector& A, const Vector& B)
{ return A.x * B.y - A.y * B.x; }
Vector Rotate(const Vector& A, double rad) {
    return Vector(A.x*cos(rad) - A.y*sin(rad), A.x*sin(rad) + A.y*cos(rad));
}
Vector Normal(const Vector& A) {
    double L = Length(A);
    return Vector(-A.y / L, A.x / L);
}
Point GetLineIntersection(const Point& P, const Point& v,
    const Point& Q, const Point& w) {
    Vector u = P - Q;
    double t = Cross(w, u) / Cross(v, w);
    return P + v * t;
}
Point GetLineProjection(const Point& P, const Point& A, const Point& B) {
    Vector v = B - A;
    return A + v * (Dot(v, P - A) / Dot(v, v));
}
double DistanceToLine(const Point& P, const Point& A, const Point& B) {
    Vector v1 = B - A, v2 = P - A;
    return fabs(Cross(v1, v2)) / Length(v1); // 不取绝对值得到的是有向距离
}

struct Line {
    Point p;
    Vector v;
    Line(const Point& p, const Vector& v): p(p), v(v) {}
    Point point(double t) const {
        return p + v * t;
    }
    Line move(double d) const {
        return Line(p + Normal(v) * d, v);
    }
};

struct Circle {
    Point c;
    double r;
    Circle(const Point& c, double r): c(c), r(r) {}
    Point point(double a) const {
        return Point(c.x + cos(a) * r, c.y + sin(a) * r);
    }
};

Point GetLineIntersection(const Line& a, const Line& b) {
    return GetLineIntersection(a.p, a.v, b.p, b.v);
}

```

```

}

double angle(const Vector& v) {
    return atan2(v.y, v.x);
}

int getLineCircleIntersection(const Line& L, const Circle& C,
    double& t1, double& t2, vector<Point>& sol) {
    double a = L.v.x, b = L.p.x - C.c.x, c = L.v.y, d = L.p.y - C.c.y;
    double e = a*a + c*c, f = 2 * (a * b + c * d), g = b * b + d * d - C.r * C.r;
    double delta = f * f - 4 * e * g;          // 判别式
    if (dcmp(delta) < 0) return 0;             // 相离
    if (dcmp(delta) == 0) {                   // 相切
        t1 = t2 = -f / (2 * e); sol.push_back(L.point(t1));
        return 1;
    }
    // 相交
    t1 = (-f - sqrt(delta)) / (2 * e); sol.push_back(L.point(t1));
    t2 = (-f + sqrt(delta)) / (2 * e); sol.push_back(L.point(t2));
    return 2;
}

int getCircleCircleIntersection(const Circle& C1,
    const Circle& C2, vector<Point>& sol) {
    double d = Length(C1.c - C2.c);
    if (dcmp(d) == 0) {
        if (dcmp(C1.r - C2.r) == 0) return -1; // 重合, 无穷多交点
        return 0;
    }
    if (dcmp(C1.r + C2.r - d) < 0) return 0;
    if (dcmp(fabs(C1.r - C2.r) - d) > 0) return 0;

    double a = angle(C2.c - C1.c);
    double da = acos((C1.r * C1.r + d * d - C2.r * C2.r) / (2 * C1.r * d));
    Point p1 = C1.point(a - da), p2 = C1.point(a + da);

    sol.push_back(p1);
    if (p1 == p2) return 1;
    sol.push_back(p2);
    return 2;
}

/***** Problem 1 *****/

Circle CircumscribedCircle(const Point& p1, const Point& p2, const Point& p3)
{
    double Bx = p2.x - p1.x, By = p2.y - p1.y;
    double Cx = p3.x - p1.x, Cy = p3.y - p1.y;
    double D = 2 * (Bx * Cy - By * Cx);

```

```

    double cx = (Cy * (Bx * Bx + By * By) - By * (Cx * Cx + Cy * Cy)) / D + p1.x;
    double cy = (Bx * (Cx * Cx + Cy * Cy) - Cx * (Bx * Bx + By * By)) / D + p1.y;
    Point p = Point(cx, cy);
    return Circle(p, Length(p1 - p));
}

/***** Problem 2 *****/

Circle InscribedCircle(const Point& p1, const Point& p2, const Point& p3) {
    double a = Length(p2 - p3);
    double b = Length(p3 - p1);
    double c = Length(p1 - p2);
    Point p = (p1 * a + p2 * b + p3 * c) / (a + b + c);
    return Circle(p, DistanceToLine(p, p1, p2));
}

/***** Problem 3 *****/

// 过点p做圆C的切线。v[i]是第i条切线的向量，返回切线条数
int getTangents(const Point& p, const Circle& C, Vector* v) {
    Vector u = C.c - p;
    double dist = Length(u);
    if (dist < C.r) return 0;
    else if (dcmp(dist - C.r) == 0) { // p在圆上，只有一条切线
        v[0] = Rotate(u, PI / 2);
        return 1;
    } else {
        double ang = asin(C.r / dist);
        v[0] = Rotate(u, -ang);
        v[1] = Rotate(u, +ang);
        return 2;
    }
}

/***** Problem 4 *****/

vector<Point> CircleThroughPointTangentToLineGivenRadius(
    const Point& p, const Line& L, double r) {
    vector<Point> ans;
    double t1, t2;
    getLineCircleIntersection(L.move(-r), Circle(p, r), t1, t2, ans);
    getLineCircleIntersection(L.move(r), Circle(p, r), t1, t2, ans);
    return ans;
}

/***** Problem 5 *****/

vector<Point> CircleTangentToLinesGivenRadius(
    const Line& a, const Line& b, double r) {

```

```

vector<Point> ans;
Line L1 = a.move(-r), L2 = a.move(r);
Line L3 = b.move(-r), L4 = b.move(r);
ans.push_back(GetLineIntersection(L1, L3));
ans.push_back(GetLineIntersection(L1, L4));
ans.push_back(GetLineIntersection(L2, L3));
ans.push_back(GetLineIntersection(L2, L4));
return ans;
}

/***** Problem 6 *****/

vector<Point> CircleTangentToTwoDisjointCirclesWithRadius(
    const Circle& c1, const Circle& c2, double r) {
    vector<Point> ans;
    Vector v = c2.c - c1.c;
    double dist = Length(v);
    int d = dcmp(dist - c1.r - c2.r - r * 2);
    if (d > 0) return ans;
    getCircleCircleIntersection(Circle(c1.c, c1.r+r), Circle(c2.c, c2.r+r), ans);
    return ans;
}

// 規整化
double lineAngleDegree(const Vector& v) {
    double ang = angle(v) * 180.0 / PI;
    while (dcmp(ang) < 0) ang += 360.0;
    while (dcmp(ang - 180) >= 0) ang -= 180.0;
    return ang;
}

void format(Circle c) {
    printf("%.6lf,%.6lf,%.6lf\n", c.c.x, c.c.y, c.r);
}

void format(vector<double> ans) {
    int n = ans.size();
    sort(ans.begin(), ans.end());
    printf("[");
    if (n) {
        printf("%.6lf", ans[0]);
        for (int i = 1; i < n; i++) printf(",%.6lf", ans[i]);
    }
    printf("]\n");
}

void format(vector<Point> ans) {
    int n = ans.size();
    sort(ans.begin(), ans.end());

```

```

printf("[");
if (n) {
    printf("%.6lf,%.6lf", ans[0].x, ans[0].y);
    for (int i = 1; i < n; i++)
        printf(",(%.6lf,%.6lf)", ans[i].x, ans[i].y);
}
printf("]\n");
}

Line getLine(const Point& p1, const Point& p2) {
    return Line(p1, p2 - p1);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    for (string cmd; cin >> cmd;) {
        double r1, r2, r;
        Point p1, p2, p3, p4, pc, pp;
        if (cmd == "CircumscribedCircle") {
            cin >> p1 >> p2 >> p3;
            format(CircumscribedCircle(p1, p2, p3));
        }
        if (cmd == "InscribedCircle") {
            cin >> p1 >> p2 >> p3;
            format(InscribedCircle(p1, p2, p3));
        }
        if (cmd == "TangentLineThroughPoint") {
            cin >> pc >> r >> pp;
            Vector v[2];
            vector<double> ans;
            int cnt = getTangents(pp, Circle(pc, r), v);
            for (int i = 0; i < cnt; i++) ans.push_back(lineAngleDegree(v[i]));
            format(ans);
        }
        if (cmd == "CircleThroughAPointAndTangentToALineWithRadius") {
            cin >> pp >> p1 >> p2 >> r;
            format(CircleThroughPointTangentToLineGivenRadius(pp, getLine(p1, p2), r));
        }
        if (cmd == "CircleTangentToTwoLinesWithRadius") {
            cin >> p1 >> p2 >> p3 >> p4 >> r;
            format(CircleTangentToLinesGivenRadius
                (getLine(p1, p2), getLine(p3, p4), r));
        }
        if (cmd == "CircleTangentToTwoDisjointCirclesWithRadius") {
            cin >> p1 >> r1 >> p2 >> r2 >> r;
            Circle c1(p1, r1), c2(p2, r2);
            format(CircleTangentToTwoDisjointCirclesWithRadius(c1, c2, r));
        }
    }
    return 0;
}

```



```
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.2.1 节例题 4
注意：本题封装了 Point 的“>>”操作，可大幅度简化代码
同类问题：Fighting against a Polygonal Monster, UVa 11177
*/
```

例6-4 【圆的综合问题】圆盘问题（Viva Confetti, Kanazawa 2002, UVa 1308）

假设已经把 n ($1 \leq n \leq 100$) 个圆盘依次放到了桌面上。现按照放置顺序依次给出各个圆盘的圆心位置和半径，问最后有多少圆盘可见（见图6-3）。

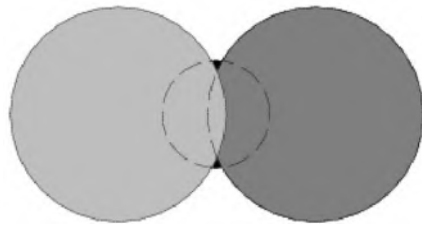


图6-3 圆盘问题

【代码实现】

```

// 刘汝佳
#include <cmath>
#include <iostream>
#include <vector>
#include <set>
#include <algorithm>
using namespace std;

const double eps = 5 * 1e-13;
int dcmp(double x) {
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}
const double PI = acos(-1), TWO_PI = PI * 2;

double NormalizeAngle(double rad, double center = PI) {
    return rad - TWO_PI * floor((rad + PI - center) / TWO_PI);
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0): x(x), y(y) {}
};

istream& operator>>(istream& is, Point& p)
{ return is >> p.x >> p.y; }

typedef Point Vector;
Vector operator + (const Vector& A, const Vector& B)
{ return Vector(A.x + B.x, A.y + B.y); }
Vector operator - (const Point&A, const Point&B)
{ return Vector(A.x - B.x, A.y - B.y); }
Vector operator * (const Vector& A, double p)
{ return Vector(A.x * p, A.y * p); }
Vector operator / (const Vector& A, double p)
{ return Vector(A.x / p, A.y / p); }
double Dot(const Vector& A, const Vector& B)

```

```

{ return A.x * B.x + A.y * B.y; }
double Length(const Vector& A) { return sqrt(Dot(A, A)); }
double angle(const Vector& v) { return atan2(v.y, v.x); }

// 交点相对于圆 1 的极角保存在 rad 中
void getCircleCircleIntersection(const Point&c1, double r1,
                                const Point&c2, double r2, vector<double>& rad) {
    double d = Length(c1 - c2);
    if (dcmp(d) == 0) return; // 不管是内含还是重合, 都不相交
    if (dcmp(r1 + r2 - d) < 0) return;
    if (dcmp(fabs(r1 - r2) - d) > 0) return;
    double a = angle(c2 - c1),
           da = acos((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d));
    rad.push_back(NormalizeAngle(a - da));
    rad.push_back(NormalizeAngle(a + da));
}

const int maxn = 100 + 5;
int N;
Point center[maxn];
double radius[maxn];
bool vis[maxn];

// 覆盖点 p 的最上层的圆
int topmost(const Point& p) {
    for (int i = N - 1; i >= 0; i--)
        if (Length(center[i] - p) < radius[i]) return i;
    return -1;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    while (cin >> N && N) {
        for (int i = 0; i < N; i++) cin >> center[i] >> radius[i];
        fill_n(vis, N + 1, 0);
        for (int i = 0; i < N; i++) {
            vector<double> rad;
            // 考虑圆 i 被切割成的各个圆弧。把圆周当作区间来处理, 起点是 0, 终点是 2PI
            rad.push_back(0), rad.push_back(TWO_PI);
            for (int j = 0; j < N; j++)
                getCircleCircleIntersection(center[i], radius[i], center[j], radius[j], rad);
            sort(rad.begin(), rad.end());

            for (size_t j = 0; j < rad.size(); j++) {
                double mid = (rad[j] + rad[j + 1]) / 2.0; // 圆弧中点相对于圆 i 圆心的极角
                for (int side = -1; side <= 1; side += 2) {
                    double r2 = radius[i] - side * eps; // 往里面或者外面稍微移动一点点
                    int t = topmost(Point(center[i].x + cos(mid) * r2, center[i].y + sin(mid)
* r2));
                    if (t >= 0) vis[t] = true;
                }
            }
        }
    }
}

```